llnl_c1/lulesh · lulesh.cc · llm_pragmas_v5_exact · claude-haiku-4-5-20251001 · FASTER

−96 / +230 lines of 1967 · 22 parallel region(s) added · 29 of 49 candidate(s) kept · RESTRUCTURING (177 code line(s) added, 91 removed) · restructuring level 0

BEFORE

```
65                                  const Index_t* elemToNode,
66                                  Real_t elemX[8],
67                                  Real_t elemY[8],
68                                  Real_t elemZ[8])
69  {
70     Index_t nd0i = elemToNode[0] ;
71     Index_t nd1i = elemToNode[1] ;
72     Index_t nd2i = elemToNode[2] ;
73     Index_t nd3i = elemToNode[3] ;
74     Index_t nd4i = elemToNode[4] ;
75     Index_t nd5i = elemToNode[5] ;
76     Index_t nd6i = elemToNode[6] ;
77     Index_t nd7i = elemToNode[7] ;
78
79     elemX[0] = domain.x(nd0i);
80     elemX[1] = domain.x(nd1i);
81     elemX[2] = domain.x(nd2i);
82     elemX[3] = domain.x(nd3i);
83     elemX[4] = domain.x(nd4i);
84     elemX[5] = domain.x(nd5i);
85     elemX[6] = domain.x(nd6i);
86     elemX[7] = domain.x(nd7i);
87
88     elemY[0] = domain.y(nd0i);
89     elemY[1] = domain.y(nd1i);
90     elemY[2] = domain.y(nd2i);
91     elemY[3] = domain.y(nd3i);
92     elemY[4] = domain.y(nd4i);
93     elemY[5] = domain.y(nd5i);
94     elemY[6] = domain.y(nd6i);
95     elemY[7] = domain.y(nd7i);
96
97     elemZ[0] = domain.z(nd0i);
98     elemZ[1] = domain.z(nd1i);
99     elemZ[2] = domain.z(nd2i);
100    elemZ[3] = domain.z(nd3i);
101    elemZ[4] = domain.z(nd4i);
102    elemZ[5] = domain.z(nd5i);
103    elemZ[6] = domain.z(nd6i);
104    elemZ[7] = domain.z(nd7i);
105
106 }
107
108 static inline
109 void InitStressTermsForElems(Domain &domain,
    …
304 void IntegrateStressForElems( Domain &domain,
305                               Real_t *sigxx, Real_t *sigyy, Real_t *sigzz,
306                               Real_t *determ, Index_t numElem, Index_t numNode
307 {
308
309    Real_t fx_local[8] ;
310    Real_t fy_local[8] ;
311    Real_t fz_local[8] ;
312
313   for( Index_t k=0 ; k<numElem ; ++k )
314   {
315     const Index_t* const elemToNode = domain.nodelist(k);
316     Real_t B[3][8] ;
    …
656    Real_t *dvdz = Allocate<Real_t>(numElem8) ;
657    Real_t *x8n  = Allocate<Real_t>(numElem8) ;
658    Real_t *y8n  = Allocate<Real_t>(numElem8) ;
659    Real_t *z8n  = Allocate<Real_t>(numElem8) ;
660
661    for (Index_t i=0 ; i<numElem ; ++i){
662       Real_t  x1[8],  y1[8],  z1[8] ;
663       Real_t pfx[8], pfy[8], pfz[8] ;
664
665       Index_t* elemToNode = domain.nodelist(i);
    …
679       }
680
681       determ[i] = domain.volo(i) * domain.v(i);
682
683       if ( domain.v(i) <= Real_t(0.0) ) {
684          exit(VolumeError);
685       }
686    }
687
688    if ( hgcoef > Real_t(0.) ) {
689       CalcFBHourglassForceForElems( domain,
    …
716
717       IntegrateStressForElems( domain,
718                                sigxx, sigyy, sigzz, determ, numElem,
719                                domain.numNode()) ;
720
```

AFTER

```
65                                  const Index_t* elemToNode,
66                                  Real_t elemX[8],
67                                  Real_t elemY[8],
68                                  Real_t elemZ[8])
69  {
70     for (Index_t i = 0; i < 8; ++i) {
71        Index_t ndi = elemToNode[i];
72        elemX[i] = domain.x(ndi);
73        elemY[i] = domain.y(ndi);
74        elemZ[i] = domain.z(ndi);
75     }
76
77  }
78
79  static inline
80  void InitStressTermsForElems(Domain &domain,
    …
275 void IntegrateStressForElems( Domain &domain,
276                               Real_t *sigxx, Real_t *sigyy, Real_t *sigzz,
277                               Real_t *determ, Index_t numElem, Index_t numNode
278 {
279
280    Real_t fx_local[8] ;
281    Real_t fy_local[8] ;
282    Real_t fz_local[8] ;
283
284    for( Index_t k=0 ; k<numElem ; ++k )
285    {
286      const Index_t* const elemToNode = domain.nodelist(k);
287      Real_t B[3][8] ;
    …
627    Real_t *dvdz = Allocate<Real_t>(numElem8) ;
628    Real_t *x8n  = Allocate<Real_t>(numElem8) ;
629    Real_t *y8n  = Allocate<Real_t>(numElem8) ;
630    Real_t *z8n  = Allocate<Real_t>(numElem8) ;
631
632    #pragma omp parallel for shared(domain, numElem, dvdx, dvdy, dvdz, x8n, y8n
633    for (Index_t i=0 ; i<numElem ; ++i){
634       Real_t  x1[8],  y1[8],  z1[8] ;
635       Real_t pfx[8], pfy[8], pfz[8] ;
636
637       Index_t* elemToNode = domain.nodelist(i);
    …
651       }
652
653       determ[i] = domain.volo(i) * domain.v(i);
654
655       if ( domain.v(i) <= Real_t(0.0) ) {
656          #pragma omp critical
657          {
658             exit(VolumeError);
659          }
660       }
661    }
662
663    if ( hgcoef > Real_t(0.) ) {
664       CalcFBHourglassForceForElems( domain,
    …
691
692       IntegrateStressForElems( domain,
693                                sigxx, sigyy, sigzz, determ, numElem,
694                                domain.numNode()) ;
695
696       // Inline and parallelize hourglass control loop to increase work granul
697       Index_t numElem8 = numElem * 8 ;
698       Real_t *dvdx = Allocate<Real_t>(numElem8) ;
```

red = removed green = added yellow = parallel region (#pragma omp) right strip = the whole file: green where changed, yellow where a parallel region now is